FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES

CS054
(INTELLIGENT SYSTEMS)

EXERCISE

3

Data Mining

Group Name:	Joshua Emmanuel A. Guerrero
Members:	**Name** / **Role** (empty)
Section:	AN41
Professor:	Ms. Tan

I. PROGRAM OUTCOME/S (PO) ADDRESSED BY THE LABORATORY EXERCISE

- Design and evaluate solutions for computing problems. [PO: e]

II. COURSE LEARNING OUTCOME/S (CLO) ADDRESSED BY THE LABORATORY EXERCISE

- To design an Intelligent System based on methods learned. [CLO: 3]

III. INTENDED LEARNING OUTCOME/S (ILO) OF THE LABORATORY EXERCISE

At the end of this exercise, students must be able to:

- Demonstrate understanding of the following algorithms: Decision Tree, K-Mean, Apriori.
- Implement Decision Tree Algorithm by designing and developing a simple predictor program using Python.

IV. BACKGROUND INFORMATION

In order to accomplish this exercise, the student must have a clear understanding of the following topics:

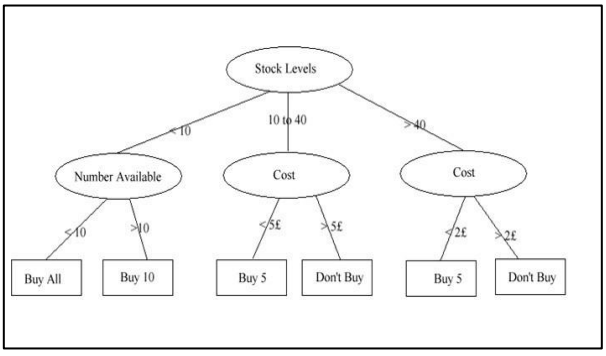
Decision Tree Algorithm

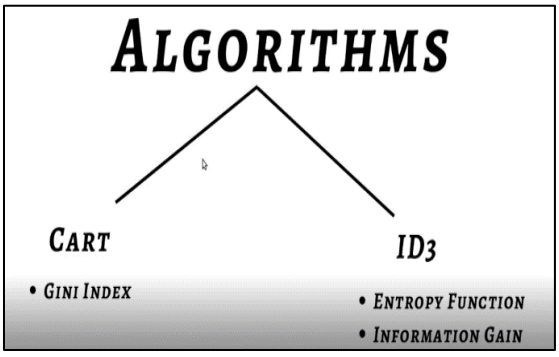
- Related to most of the other techniques (primarily classification and prediction), the decision tree can be used either as a part of the selection criteria, or to support the use and selection of specific data within the overall structure.

A Decision Tree is a tree:

- where each node represents a feature (attribute)
- each branch represents a decision (rule)
- each leaf represents an outcome

Example of Decision Tree





https://octaviansima.wordpress.com/2011/03/25/decision-trees-c4-5/

What is an ID3 Algorithm?

- Stands for Iterative Dichotomiser 3
- An algorithm developed by J.Ross Quinlan
- The core algorithm for building decision trees.
- Supervised learning algorithm used for classification problems.
- It is a classification algorithm that follows a greedy approach by selecting a best attribute that yields maximum Information Gain(IG) or minimum Entropy(H).

STEPS in Creating the Decision Tree
Important Concepts:

- Calculate **Entropy (the amount of uncertainty in dataset : the number of positive and negative evidences).**

$$Entropy = \frac{-p}{p+n} log_2(\frac{p}{p+n}) - \frac{n}{p+n} log_2(\frac{n}{p+n})$$

** Entropy(S) – the entropy of the entire data set.
** Computing the entropy of each attribute

- Calculate **Average Information**.

$$I(Attribute) = \sum \frac{p_i + n_i}{p+n} Entropy(A)$$

- Calculate **Information Gain** (Difference in Entropy before and after splitting dataset on attribute A)

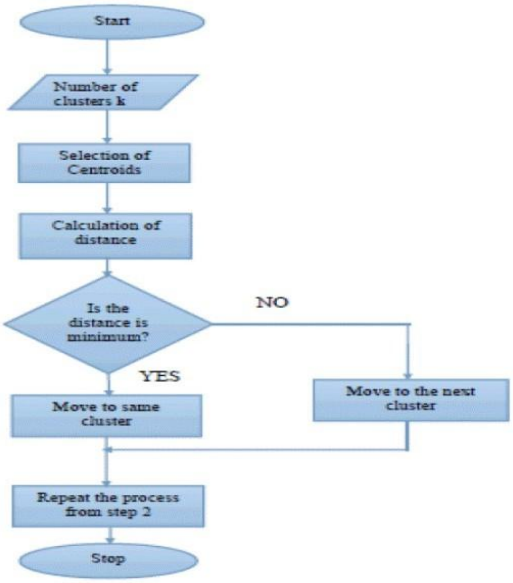
$$Gain = Entropy(S) - I(Attribute)$$

K-Means Algorithm

K-means creates k groups from a set of objects so that the members of a group are more similar. It's a popular cluster analysis technique for exploring a dataset.
Cluster analysis is a family of algorithms designed to form groups such that the group members are more similar versus non-group members. Clusters and groups are synonymous in the world of cluster analysis.

K-means Clustering Algorithm

1. Choose a value of k number of clusters to be formed.
2. Randomly select k data points from the data set as the initial cluster centroids/centers
3. For each datapoint:
   Compute the distance between thedatapoint and the cluster centroid
   Assign the datapoint to the closest centroid
4. 4. For each cluster calculate the new mean based on the datapoints in the cluster.
5. 5. Repeat 3 & 4 steps until mean of the clusters stops changing or maximum number of iterations reached.



https://miro.medium.com/max/1022/1*ZN6cDw34wEO-dxydDwY4vA.png

Apriori Algorithm

What does it do?
It learns association rules and is applied to a database containing a large number of transactions.

What is Association Rule?
It is a data mining technique for learning correlations and relations among variables in a database.

Apriori – Example
In a database full of supermarket transactions, think of a database as a giant spreadsheet where each row is a customer transaction and every column represents a different grocery item.

Transaction ID	Chips	Dip	Soda	Apples	Milk
1	X	X	X		
2	X	X			X
3	X		X		

By applying the Apriori algorithm, we can learn the grocery items that are purchased together a.k.a association rule. One can find those items that tend to be purchased together more frequently than other items — the ultimate goal being to get shoppers to buy more. Together, these items are called “itemsets”.

How Apriori works?
Define first the size of itemset. Do you want to see patterns for a 2-itemset, etc.?
The second is support or the number of transactions containing the itemset divided by the total number of transactions. An itemset that meets the support is called a frequent itemset.

The third is your confidence or the conditional probability of some item, given you have certain other items in your itemset.
A good example is, given chips is in your itemset, there is a 67% confidence of having soda also in the itemset.

V. GRADING SYSTEM / RUBRIC (please see separate sheet)

VI. LABORATORY ACTIVITY

In this problem, you will come up with a decision tree to predict if a planet is habitable based only on features observed by the telescope.

In the table below, you are given the data from all 800 planets surveyed so far. The features observed by telescope are Size ("Big" or "Small"), and Orbit ("Near" or "Far"). Each row indicates the values of the features and habitability, and how many times that set of values was observed. So, for example, there were 20 "Big" planets "Near" their star that were habitable.

Size	Orbit	Habitable	Count
Big	Near	Yes	20
Big	Far	Yes	170
Small	Near	Yes	139
Small	Far	Yes	45
Big	Near	No	130
Big	Far	No	30
Small	Near	No	11
Small	Far	No	255

a. Determine if Planet=Big and Orbit=Far is habitable? Derive and draw the decision tree learned by ID3 on this data.

b. Determine if Planet=Small and Orbit=Near is habitable? Derive and draw the decision tree learned by ID3 on this data.

Derive a decision tree learned by ID3 on this data (use the maximum information gain criterion for splits, don't do any pruning). Make sure to clearly mark at each node what attribute you are splitting on, and which value corresponds to which branch. By each leaf node of the tree, write in the number of habitable and inhabitable planets in the training data that belong to that node.

OUTPUT:

```
PS C:\Users\Joshua\Downloads\python\technicals> & "C:\Program Files\
y
Total planets: 800
Initial entropy: 0.996950 bits
Information gain (Size): 0.012820
Information gain (Orbit): 0.006790
Information gain (Orbit | Size=Big): 0.403424
Information gain (Orbit | Size=Small): 0.443273

Split: Size
  [Big]
    Split: Orbit
      [Near]
        Habitable: No (Yes=20, No=130)
      [Far]
        Habitable: Yes (Yes=170, No=30)
  [Small]
    Split: Orbit
      [Near]
        Habitable: Yes (Yes=139, No=11)
      [Far]
        Habitable: No (Yes=45, No=255)

Size=Big, Orbit=Far -> Habitable: Yes

Size=Small, Orbit=Near -> Habitable: Yes

Diagram saved to: decision_tree.svg
PS C:\Users\Joshua\Downloads\python\technicals>
```

CODE:

```
from collections import Counter
from html import escape
from math import log2
from pathlib import Path

# Size, Orbit, Habitable, Count (the eight rows represent 800 planets).
DATA = [
    {"Size": "Big", "Orbit": "Near", "Habitable": "Yes", "Count": 20},
    {"Size": "Big", "Orbit": "Far", "Habitable": "Yes", "Count": 170},
    {"Size": "Small", "Orbit": "Near", "Habitable": "Yes", "Count": 139},
    {"Size": "Small", "Orbit": "Far", "Habitable": "Yes", "Count": 45},
    {"Size": "Big", "Orbit": "Near", "Habitable": "No", "Count": 130},
    {"Size": "Big", "Orbit": "Far", "Habitable": "No", "Count": 30},
    {"Size": "Small", "Orbit": "Near", "Habitable": "No", "Count": 11},
    {"Size": "Small", "Orbit": "Far", "Habitable": "No", "Count": 255},
]

def class_counts(rows):
    counts = Counter()
    for row in rows:
        counts[row["Habitable"]] += row["Count"]
    return counts

def entropy(rows):
    counts = class_counts(rows)
    total = sum(counts.values())
    return -sum((n / total) * log2(n / total) for n in counts.values() if n)

def partitions(rows, attribute):
    groups = {}
    for row in rows:
        groups.setdefault(row[attribute], []).append(row)
    return groups

def information_gain(rows, attribute):
    total = sum(row["Count"] for row in rows)
    remainder = sum(
        sum(row["Count"] for row in group) / total * entropy(group)
```

```
            for group in partitions(rows, attribute).values()
        )
        return entropy(rows) - remainder

    def id3(rows, attributes):
        counts = class_counts(rows)
        # Majority vote labels mixed leaves when no attributes remain.
        prediction = max(("Yes", "No"), key=lambda label: counts[label])
        node = {"counts": counts, "prediction": prediction}
        if len([n for n in counts.values() if n]) == 1 or not attributes:
            return node
        best = max(attributes, key=lambda attribute: information_gain(rows, attribute))
        node["attribute"] = best
        node["children"] = {
            value: id3(group, [attribute for attribute in attributes if attribute != best])
            for value, group in partitions(rows, best).items()
        }
        return node

    def predict(tree, sample):
        node = tree
        while "attribute" in node:
            value = sample[node["attribute"]]
            node = node["children"][value]
        return node["prediction"]

    def print_tree(node, indent=""):
        if "attribute" not in node:
            print(f"{indent}Habitable: {node['prediction']} "
                  f"(Yes={node['counts']['Yes']}, No={node['counts']['No']})")
            return
        print(f"{indent}Split: {node['attribute']}")
        for value, child in node["children"].items():
            print(f"{indent}  [{value}]")
            print_tree(child, indent + "    ")

    def save_tree_svg(tree, path):
        """Draw the learned tree as an editable, dependency-free SVG."""
        leaves = []
```

```
nodes = []
edges = []

def layout(node, depth):
    children = []
    for label, child in node.get("children", {}).items():
        children.append((label, layout(child, depth + 1)))
    if children:
        x = sum(child[0] for _, child in children) / len(children)
    else:
        x = 160 + 300 * len(leaves)
        leaves.append(node)
    y = 145 + depth * 170
    for label, child in children:
        edges.append((x, y, child[0], child[1], label))
    nodes.append((x, y, node))
    return x, y

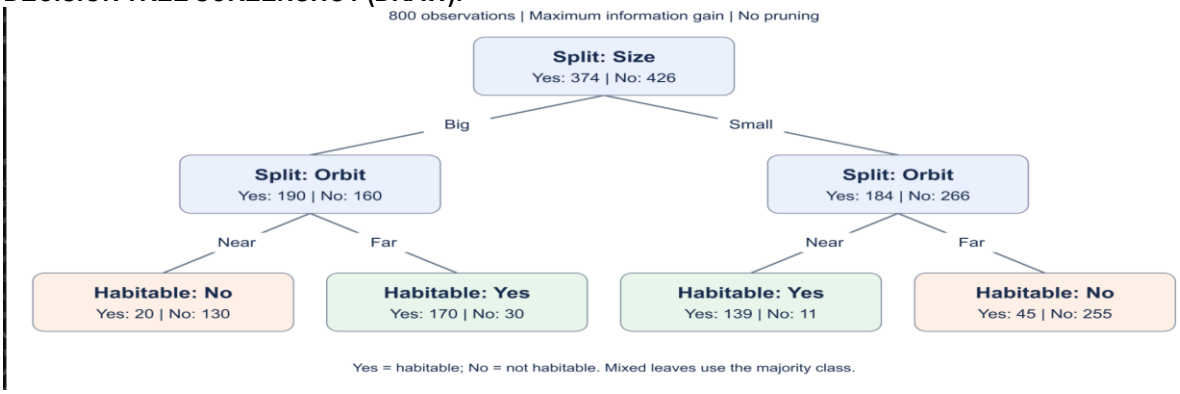
layout(tree, 0)
width = max(640, 300 * len(leaves) + 20)
height = max(y for _, y, _ in nodes) + 130
svg = [f'<svg xmlns="http://www.w3.org/2000/svg" width="{width}" height="{height}" '
       f'viewBox="0 0 {width} {height}">',
       '<rect width="100%" height="100%" fill="white"/>',
       '<g font-family="Arial, sans-serif" text-anchor="middle" fill="#172b4d">',
       f'<text x="{width/2}" y="45" font-size="28" font-weight="bold">'
       'Technical 7: Planet Habitability - ID3</text>',
       f'<text x="{width/2}" y="78" font-size="17">'
       '800 observations | Maximum information gain | No pruning</text>']
for x, y, cx, cy, label in edges:
    svg.append(f'<path d="M {x} {y+42} L {cx} {cy-42}" stroke="#65758b" '
               'stroke-width="2" fill="none"/>')
    mx, my = (x + cx) / 2, (y + cy) / 2
    svg.append(f'<rect x="{mx-34}" y="{my-15}" width="68" height="28" fill="white"/>')
    svg.append(f'<text x="{mx}" y="{my+5}" font-size="18">{escape(label)}</text>')
for x, y, node in nodes:
    internal = "attribute" in node
    color = "#eaf0fc" if internal else ("#e7f5eb" if node["prediction"] == "Yes" else "#fff0e7")
    title = f'Split: {node["attribute"]}' if internal else f'Habitable: {node["prediction"]}'
```

```
            svg.append(f'<rect x="{x-133}" y="{y-42}" width="266" height="84" rx="12" '
                       f'fill="{color}" stroke="#65758b"/>')
            svg.append(f'<text x="{x}" y="{y-6}" font-size="22" font-weight="bold">{escape(title)}</text>')
            svg.append(f'<text x="{x}" y="{y+23}" font-size="18">'
                       f'Yes: {node["counts"]["Yes"]} | No: {node["counts"]["No"]}</text>')
        svg.append(f'<text x="{width/2}" y="{height-30}" font-size="16">'
                   'Yes = habitable; No = not habitable. Mixed leaves use the majority class.</text>')
        svg.append('</g></svg>')
        path.write_text("\n".join(svg), encoding="utf-8")

    def main():
        print(f"Total planets: {sum(row['Count'] for row in DATA)}")
        print(f"Initial entropy: {entropy(DATA):.6f} bits")
        for attribute in ("Size", "Orbit"):
            print(f"Information gain ({attribute}): {information_gain(DATA, attribute):.6f}")
        tree = id3(DATA, ["Size", "Orbit"])
        for size, group in partitions(DATA, "Size").items():
            print(f"Information gain (Orbit | Size={size}): {information_gain(group, 'Orbit'):.6f}")
        print()
        print_tree(tree)
        for size, orbit in (("Big", "Far"), ("Small", "Near")):
            result = predict(tree, {"Size": size, "Orbit": orbit})
            print(f"\nSize={size}, Orbit={orbit} -> Habitable: {result}")
        path = Path(__file__).with_name("decision_tree.svg")
        save_tree_svg(tree, path)
        print(f"\nDiagram saved to: {path.name}")

    if __name__ == "__main__":
        main()
```

DECISION TREE SCREENSHOT (DRAW):



VII. QUESTION AND ANSWER

Given the details below, apply K-means algorithm to generate iteration 2 and 3. Show your solution.

We will apply k-means on the following 1 dimensional data set for K=2.
Data set {2, 4, 10, 12, 3, 20, 30, 11, 25}

Datapoint	D1	D2	Cluster
2	2	9	C1
4	0	7	C1
10	6	1	C2
12	8	1	C2
3	1	8	C1
20	16	9	C2
30	26	19	C2
11	7	0	C2
25	21	14	C2

Iteration 1
M1, M2 are the two randomly selected centroids/means where
M1= 4, M2=11
and the initial clusters are
C1= {4}, C2= {11}

OUTPUT:

```
Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\Joshua\Downloads\python\technicals> & "C:\Program Files\P
Data: [2, 4, 10, 12, 3, 20, 30, 11, 25]
Iteration 1 uses the worksheet's initial centroids 4 and 11.

ITERATION 1
Starting means: M1=4.000000, M2=11.000000
 Point           D1           D2   Cluster
     2     2.000000     9.000000        C1
     4     0.000000     7.000000        C1
    10     6.000000     1.000000        C2
    12     8.000000     1.000000        C2
     3     1.000000     8.000000        C1
    20    16.000000     9.000000        C2
    30    26.000000    19.000000        C2
    11     7.000000     0.000000        C2
    25    21.000000    14.000000        C2
C1 = [2, 4, 3]
New M1 = 9/3 = 3.000000
C2 = [10, 12, 20, 30, 11, 25]
New M2 = 108/6 = 18.000000

ITERATION 2
Starting means: M1=3.000000, M2=18.000000
 Point           D1           D2   Cluster
     2     1.000000    16.000000        C1
     4     1.000000    14.000000        C1
    10     7.000000     8.000000        C1
    12     9.000000     6.000000        C2
     3     0.000000    15.000000        C1
    20    17.000000     2.000000        C2
    30    27.000000    12.000000        C2
    11     8.000000     7.000000        C2
    25    22.000000     7.000000        C2
C1 = [2, 4, 10, 3]
New M1 = 19/4 = 4.750000
C2 = [12, 20, 30, 11, 25]
```

```
ITERATION 3
Starting means: M1=4.750000, M2=19.600000
 Point           D1           D2   Cluster
     2     2.750000    17.600000        C1
     4     0.750000    15.600000        C1
    10     5.250000     9.600000        C1
    12     7.250000     7.600000        C1
     3     1.750000    16.600000        C1
    20    15.250000     0.400000        C2
    30    25.250000    10.400000        C2
    11     6.250000     8.600000        C1
    25    20.250000     5.400000        C2
C1 = [2, 4, 10, 12, 3, 11]
New M1 = 42/6 = 7.000000
C2 = [20, 30, 25]
New M2 = 75/3 = 25.000000
PS C:\Users\Joshua\Downloads\python\technicals>
```

CODE:

```
from fractions import Fraction

DATA = [2, 4, 10, 12, 3, 20, 30, 11, 25]

def assign_clusters(data, means):
    clusters = [[], []]
    distances = []
    for value in data:
        d1, d2 = (abs(value - mean) for mean in means)
        index = 0 if d1 <= d2 else 1  # A tie goes to C1.
        clusters[index].append(value)
        distances.append((value, d1, d2, index + 1))
    return clusters, distances

def update_means(clusters, previous):
    # Retain the previous centroid if a cluster is empty.
    return [Fraction(sum(cluster), len(cluster)) if cluster else old
            for cluster, old in zip(clusters, previous)]

def main():
    means = [Fraction(4), Fraction(11)]
    print(f"Data: {DATA}")
    print("Iteration 1 uses the worksheet's initial centroids 4 and 11.")
    for iteration in range(1, 4):
        print(f"\nITERATION {iteration}")
        print(f"Starting means: M1={float(means[0]):.6f}, M2={float(means[1]):.6f}")
        clusters, distances = assign_clusters(DATA, means)
        print(f"{'Point':>6} {'D1':>12} {'D2':>12} {'Cluster':>9}")
        for value, d1, d2, cluster in distances:
```

```
            print(f"{value:>6} {float(d1):>12.6f} {float(d2):>12.6f} {'C'+str(cluster):>9}")
        new_means = update_means(clusters, means)
        for index, (cluster, mean) in enumerate(zip(clusters, new_means), 1):
            print(f"C{index} = {cluster}")
            print(f"New M{index} = {sum(cluster)}/{len(cluster)} = {float(mean):.6f}")
        if new_means == means:
            print("Centroids are unchanged: converged.")
        means = new_means

if __name__ == "__main__":
    main()
```

VIII. REFERENCES

- Decision Trees: Some Exercises
  Exemplifying the application of the ID3 algorithm in the presence of both categorical and continue attributes, CMU, 2012 fall, Eric Xing, Aarti Singh, HW1, pr. 1.1
- Li, R.. Top 10 Data Mining Algorithms, Hacker Bits. Retrieved from https://hackerbits.com/data/k-means-data-mining-algorithm/
- Shamil, F.R., 3-Mar-2020. Support, Confidence, Minimum support, Frequent itemset, K-itemset, absolute support in data mining. T4 Tutorials. Retrieved from https://t4tutorials.com/support-confidence-minimum-support-frequent-itemset-in-data-mining/
- Schott, M., 23-April-2019. K-Means Clustering Algorithm for Machine Learning. Retrieved from https://medium.com/capital-one-tech/k-means-clustering-algorithm-for-machine-learning-d1d7dc5de882